

Российский университет дружбы народов

Факультет физико-математических наук и естественных наук

Кафедра компьютерных и информационных наук

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ №6

Дисциплина: Архитектура компьютеров

Студент: *Алексеев Тимофей*
Группа: НК Абд-07-25

Москва

2025 г.

Содержание

1. Цель работы	5
2. Задание	6
3. Теоретическое введение	7
4. Выполнение лабораторной работы	9
4.1 Символические и численные данные в NASM	9
4.2 Выполнение арифметических операций в NASM	14
4.3 Ответы на контрольные вопросы	18
4.4 Задание для самостоятельной работы	19
5. Выводы	21
6. Список литературы	22

Список иллюстраций

4.1 Создание нового каталога	9
4.2 Сохранение новой программы	10
4.3 Запуск изначальной программы	11
4.4 Измененная программа	11
4.5 Запуск измененной программы	12
4.6 Вторая программа	12
4.7 Вывод второй программы	13
4.8 Вывод измененной второй программы	13
4.9 Замена функции вывода во второй программе	13
4.10 Третья программа	14
4.11 Запуск третьей программы	15
4.12 Изменение третьей программы	16
4.13 Запуск измененной третьей программы	16
4.14 Программа для подсчета варианта	17
4.15 Запуск программы для подсчета варианта	17

Список таблиц

1. Цель работы

Цель данной лабораторной работы - освоение арифметических инструкций языка ассемблера NASM.

2. Задание

1. Символьные и численные данные в NASM
2. Выполнение арифметических операций в NASM
3. Выполнение заданий для самостоятельной работы

3. Теоретическое введение

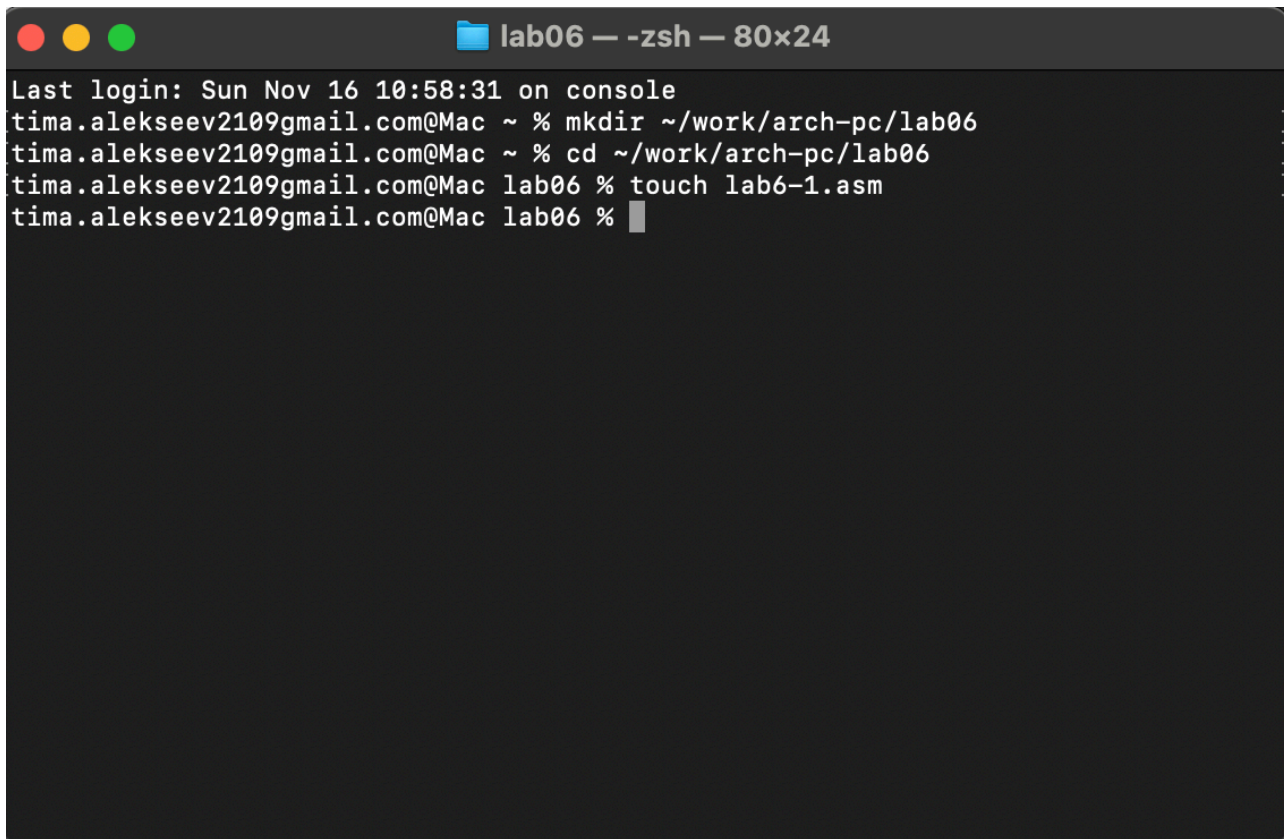
Большинство инструкций на языке ассемблера требуют обработки операндов. Адрес операнда предоставляет место, где хранятся данные, подлежащие обработке. Это могут быть данные хранящиеся в регистре или в ячейке памяти. - Регистровая адресация – операнды хранятся в регистрах и в команде используются имена этих регистров, например: `mov ax,bx`. - Непосредственная адресация – значение операнда задается непосредственно в команде, Например: `mov ax,2`. - Адресация памяти – операнд задает адрес в памяти. В команде указывается символическое обозначение ячейки памяти, над содержимым которой требуется выполнить операцию. Ввод информации с клавиатуры и вывод её на экран осуществляется в символьном виде. Кодирование этой информации производится согласно кодовой таблице символов ASCII. ASCII – сокращение от American Standard Code for Information Interchange (Американский стандартный код для обмена информацией). Согласно стандарту ASCII каждый символ кодируется одним байтом. Среди инструкций NASM нет такой, которая выводит числа (не в символьном виде). Поэтому, например, чтобы вывести число, надо предварительно преобразовать его цифры в ASCII-коды этих цифр и выводить на экран эти коды, а не само число. Если же выводить число на экран непосредственно, то экран воспримет его не как число, а как последовательность ASCII-символов – каждый байт числа будет воспринят как один ASCII-символ – и выведет на экран эти символы. Аналогичная ситуация происходит и при вводе данных с клавиатуры. Введенные данные будут представлять собой символы, что сделает невозможным получение корректного результата при выполнении над ними арифметических операций.

Для решения этой проблемы необходимо проводить преобразование ASCII символов в числа и обратно.

4. Выполнение лабораторной работы

4.1 Символьные и численные данные в NASM

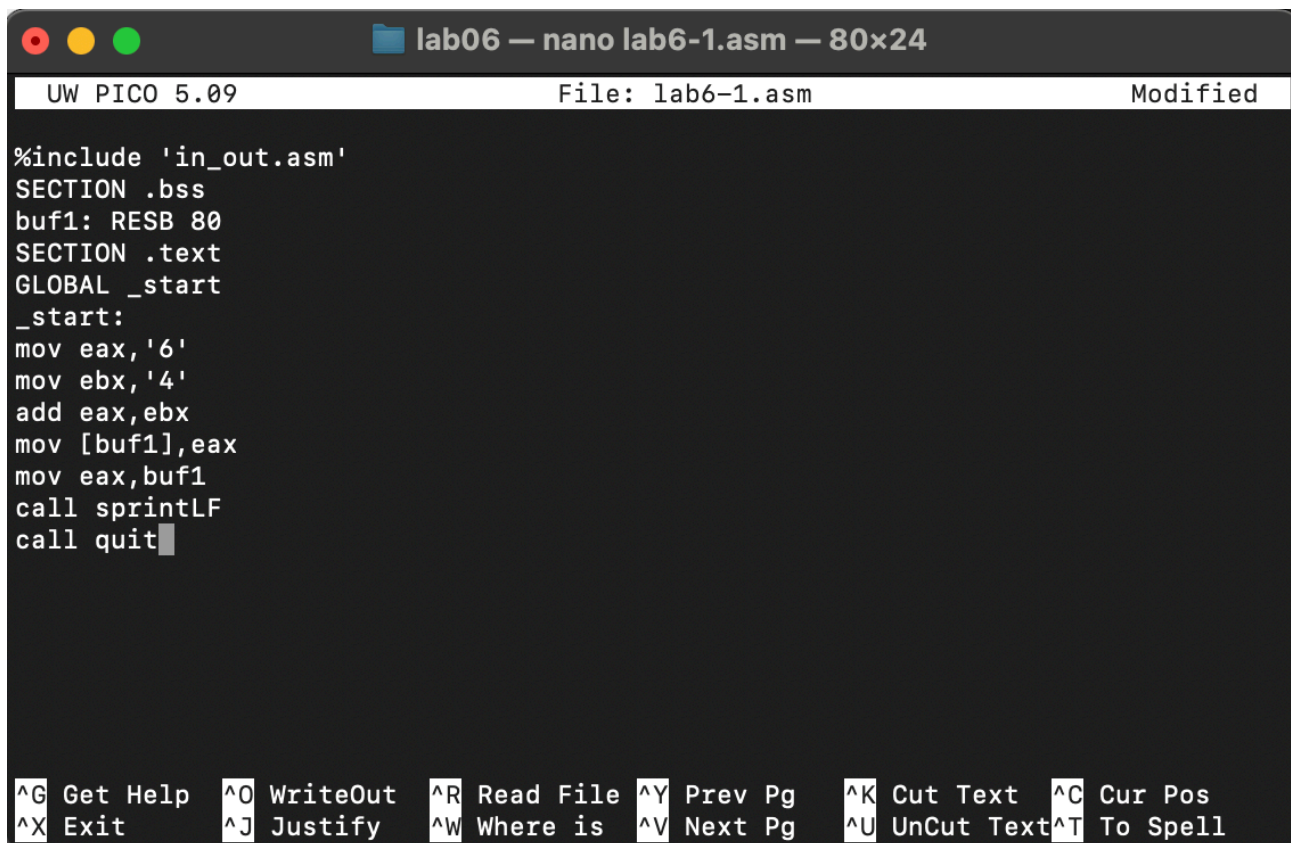
Создаю каталог для программ лабораторной работы №6 и перехожу в него, создаю там файл (рис. 4.1).

A terminal window titled 'lab06 — -zsh — 80x24' with a dark background. It shows the following commands and output:

```
Last login: Sun Nov 16 10:58:31 on console
tima.alekseev2109gmail.com@Mac ~ % mkdir ~/work/arch-pc/lab06
tima.alekseev2109gmail.com@Mac ~ % cd ~/work/arch-pc/lab06
tima.alekseev2109gmail.com@Mac lab06 % touch lab6-1.asm
tima.alekseev2109gmail.com@Mac lab06 %
```

Рис. 4.1: Создание нового каталога

В созданном файле ввожу программу из листинга (рис. 4.2).

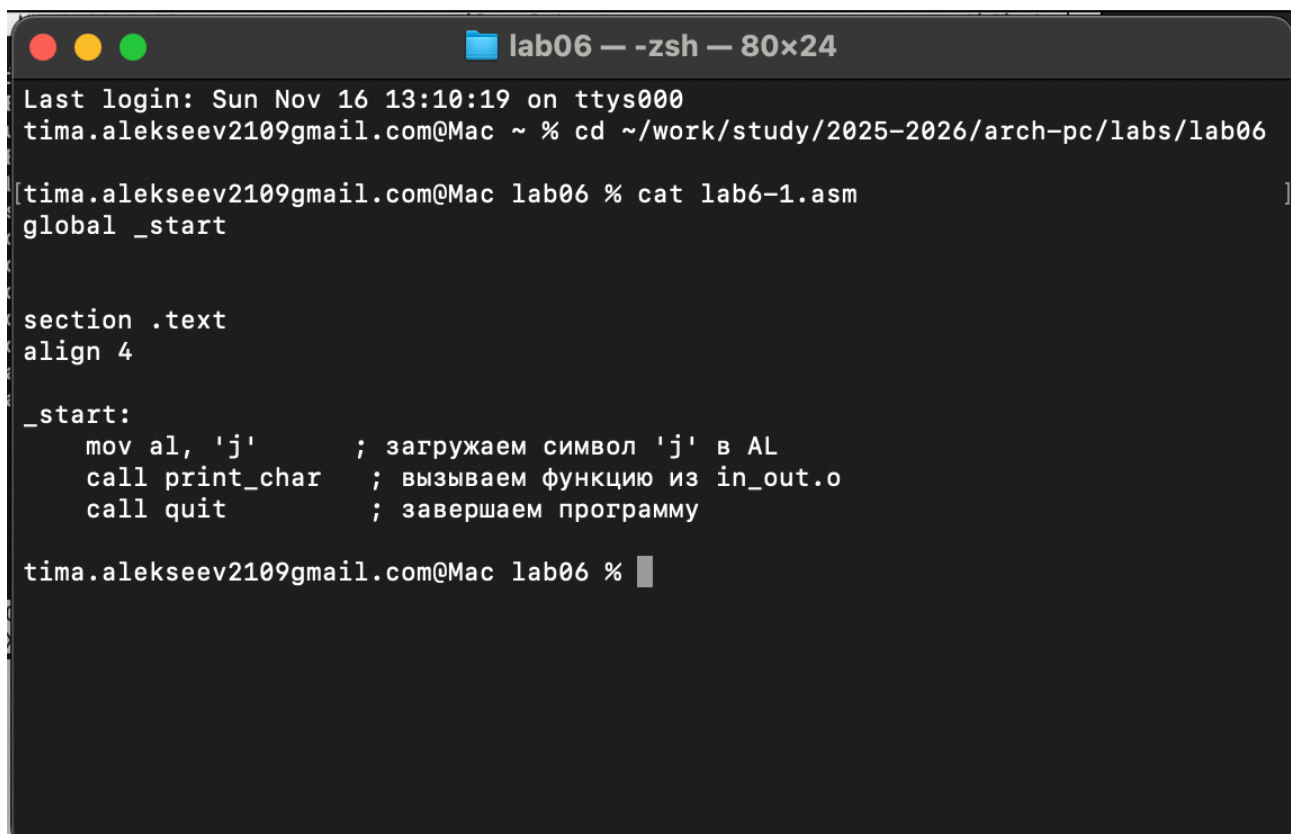


```
lab06 — nano lab6-1.asm — 80x24
UW PICO 5.09                               File: lab6-1.asm           Modified
#include 'in_out.asm'
SECTION .bss
buf1: RESB 80
SECTION .text
GLOBAL _start
_start:
mov eax, '6'
mov ebx, '4'
add eax, ebx
mov [buf1], eax
mov eax, buf1
call sprintLF
call quit
```

^G Get Help ^O WriteOut ^R Read File ^Y Prev Pg ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where is ^V Next Pg ^U UnCut Text ^T To Spell

Рис. 4.2: Сохранение новой программы

Создаю исполняемый файл и запускаю его, вывод программы отличается от предполагаемого изначально, ибо коды символов в сумме дают символ j по таблице ASCII. {#fig:003 width=70%}



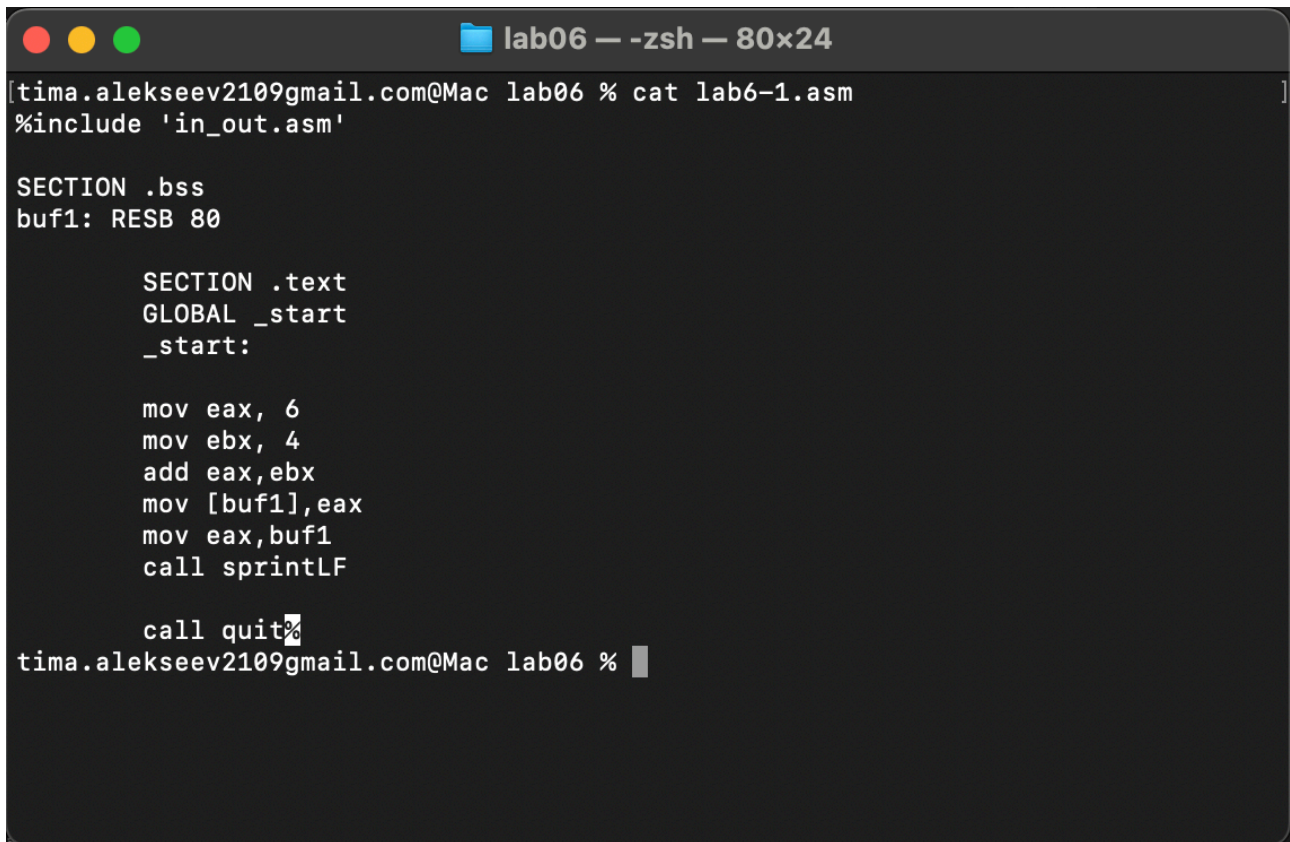
```
lab06 — -zsh — 80x24
Last login: Sun Nov 16 13:10:19 on ttys000
tima.alekseev2109gmail.com@Mac ~ % cd ~/work/study/2025-2026/arch-pc/labs/lab06
[tima.alekseev2109gmail.com@Mac lab06 % cat lab6-1.asm
global _start

section .text
align 4

_start:
    mov al, 'j'          ; загружаем символ 'j' в AL
    call print_char      ; вызываем функцию из in_out.o
    call quit            ; завершаем программу

tima.alekseev2109gmail.com@Mac lab06 %
```

Рис. 4.3: Запуск изначальной программы
Изменяю текст изначальной программы, убрав кавычки (рис.4.4)



```
lab06 — -zsh — 80x24
[tima.alekseev2109gmail.com@Mac lab06 % cat lab6-1.asm
#include 'in_out.asm'

SECTION .bss
buf1: RESB 80

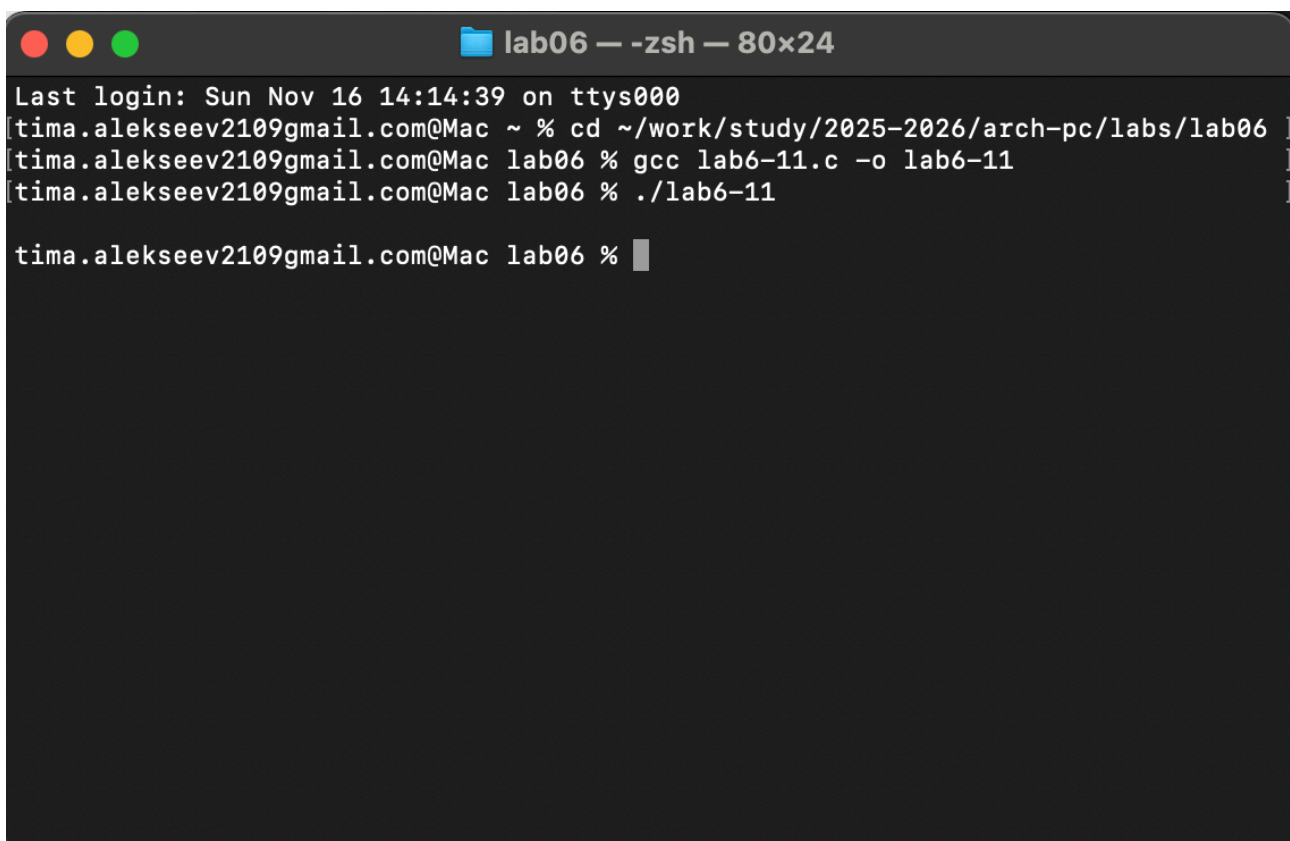
SECTION .text
GLOBAL _start
_start:

mov eax, 6
mov ebx, 4
add eax, ebx
mov [buf1], eax
mov eax, buf1
call sprintLF

call quit%
tima.alekseev2109gmail.com@Mac lab06 %
```

Рис. 4.4: Измененная программа

На этот раз программа выдала пустую строку, это связано с тем, что символ 10 означает переход на новую строку (рис. 4.5).



```
lab06 — -zsh — 80x24
Last login: Sun Nov 16 14:14:39 on ttys000
[tima.alekseev2109gmail.com@Mac ~ % cd ~/work/study/2025-2026/arch-pc/labs/lab06
[tima.alekseev2109gmail.com@Mac lab06 % gcc lab6-11.c -o lab6-11
[tima.alekseev2109gmail.com@Mac lab06 % ./lab6-11

tima.alekseev2109gmail.com@Mac lab06 %
```

Рис. 4.5: Запуск измененной программы

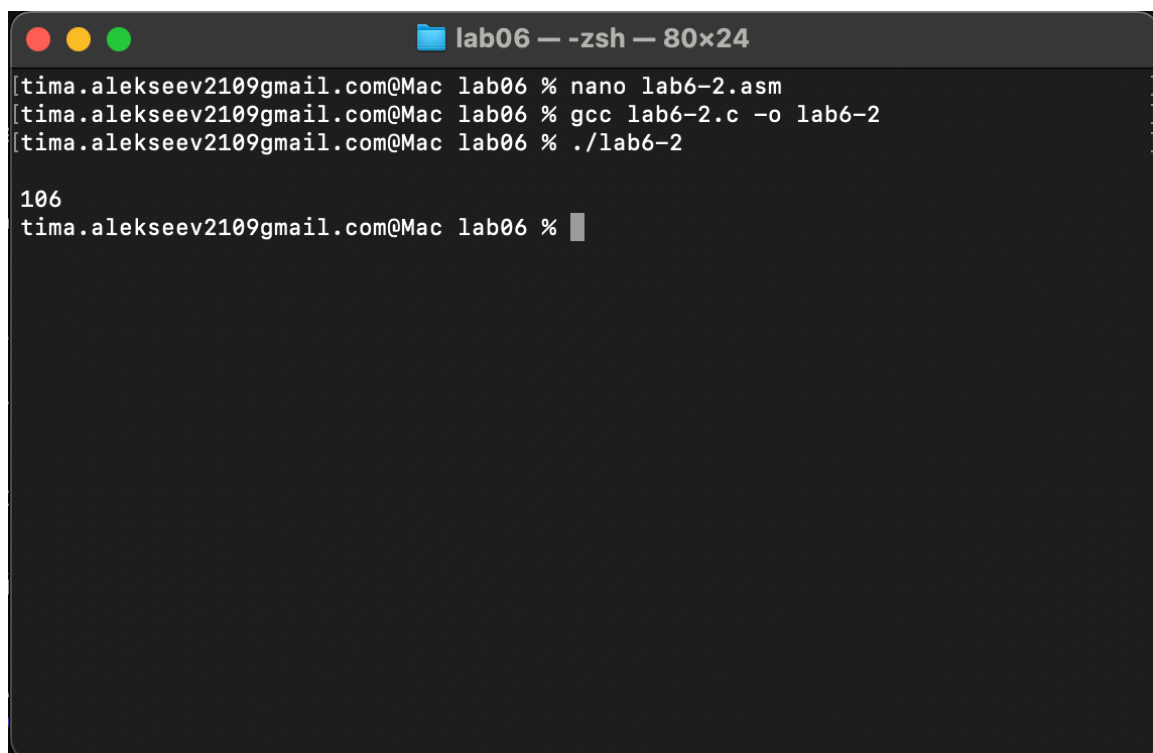
Создаю новый файл для будущей программы и записываю в нее код из листинга (рис. 4.6).



```
%include 'in_out.asm'
SECTION .text
GLOBAL _start
_start:
mov eax, '6'
mov ebx, '4'
add eax, ebx
call iprintLF
call quit
```

Рис. 4.6: Вторая программа

Создаю исполняемый файл и запускаю его, теперь отображается результат 106, программа, как и в первый раз, сложила коды символов, но вывела само число, а не его символ, благодаря замене функции вывода на iprintLF (рис. 4.7).

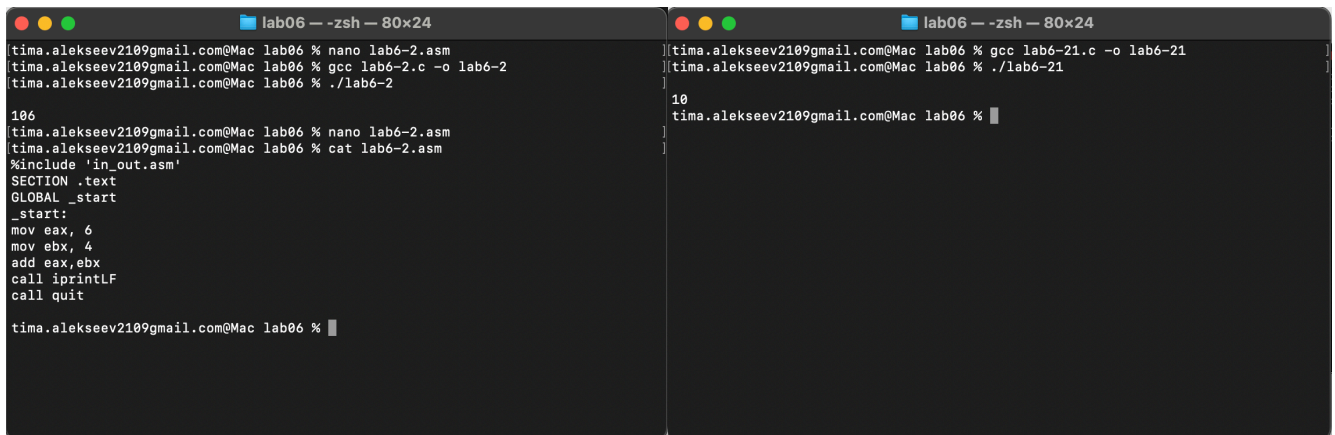


```
tima.alekseev2109gmail.com@Mac lab06 % nano lab6-2.asm
tima.alekseev2109gmail.com@Mac lab06 % gcc lab6-2.c -o lab6-2
tima.alekseev2109gmail.com@Mac lab06 % ./lab6-2

106
tima.alekseev2109gmail.com@Mac lab06 %
```


Рис. 4.7: Вывод второй программы

Убрав кавычки в программе, я снова ее запускаю и получаю предполагаемый изначально результат. (рис. 4.8).

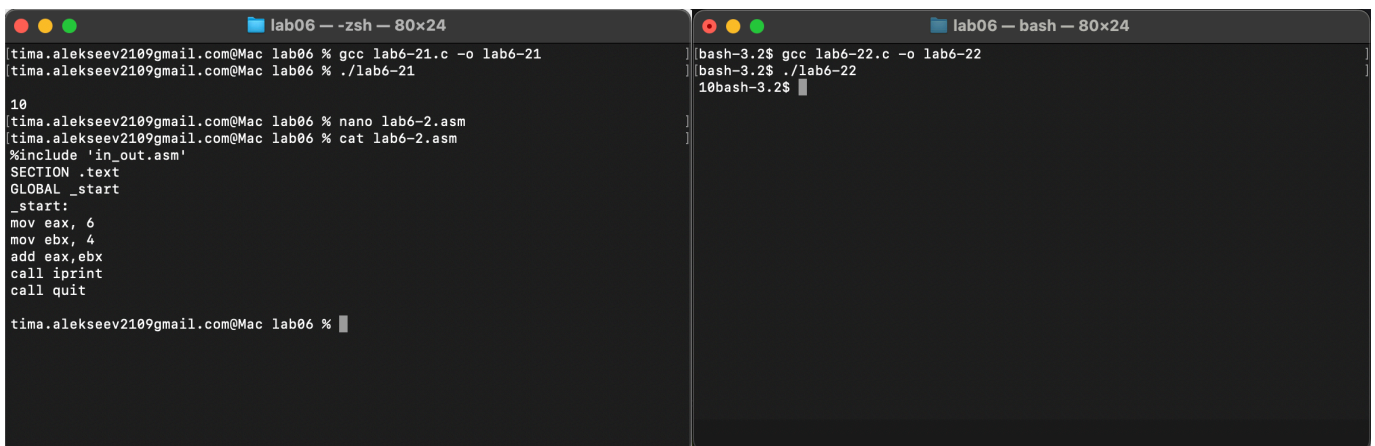


```
lab06 --zsh-- 80x24
tima.alekseev2109gmail.com@Mac lab06 % nano lab6-2.asm
tima.alekseev2109gmail.com@Mac lab06 % gcc lab6-2.c -o lab6-2
tima.alekseev2109gmail.com@Mac lab06 % ./lab6-2
10
tima.alekseev2109gmail.com@Mac lab06 % nano lab6-2.asm
tima.alekseev2109gmail.com@Mac lab06 % cat lab6-2.asm
#include 'in_out.asm'
SECTION .text
GLOBAL _start
_start:
mov eax, 6
mov ebx, 4
add eax, ebx
call iprintLF
call quit
tima.alekseev2109gmail.com@Mac lab06 %

lab06 --zsh-- 80x24
tima.alekseev2109gmail.com@Mac lab06 % gcc lab6-21.c -o lab6-21
tima.alekseev2109gmail.com@Mac lab06 % ./lab6-21
10
tima.alekseev2109gmail.com@Mac lab06 %
```

Рис. 4.8: Вывод измененной второй программы

Заменив функцию вывода на `iprint`, я получаю тот же результат, но без переноса строки (рис. 4.9).



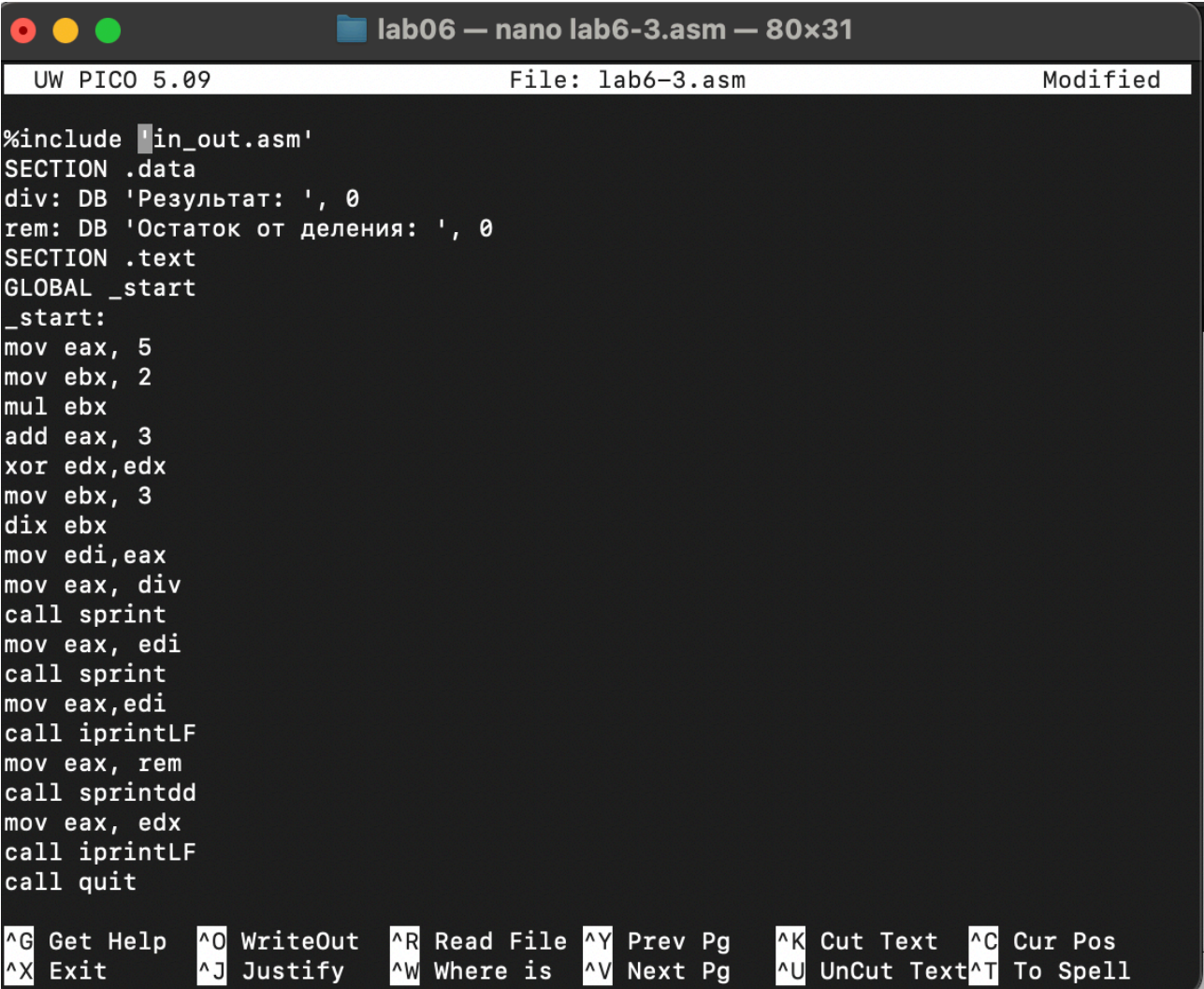
```
lab06 --zsh-- 80x24
tima.alekseev2109gmail.com@Mac lab06 % gcc lab6-21.c -o lab6-21
tima.alekseev2109gmail.com@Mac lab06 % ./lab6-21
10
tima.alekseev2109gmail.com@Mac lab06 % nano lab6-2.asm
tima.alekseev2109gmail.com@Mac lab06 % cat lab6-2.asm
#include 'in_out.asm'
SECTION .text
GLOBAL _start
_start:
mov eax, 6
mov ebx, 4
add eax, ebx
call iprint
call quit
tima.alekseev2109gmail.com@Mac lab06 %

lab06 --bash-- 80x24
[bash-3.2$ gcc lab6-22.c -o lab6-22
[bash-3.2$ ./lab6-22
10bash-3.2$
```

Рис. 4.9: Замена функции вывода во второй программе

4.2 Выполнение арифметических операций в NASM

Создаю новый файл и копирую в него содержимое листинга (рис. 4.10).



```
lab06 — nano lab6-3.asm — 80x31
UW PICO 5.09 File: lab6-3.asm Modified

%include 'in_out.asm'
SECTION .data
div: DB 'Результат: ', 0
rem: DB 'Остаток от деления: ', 0
SECTION .text
GLOBAL _start
_start:
mov eax, 5
mov ebx, 2
mul ebx
add eax, 3
xor edx,edx
mov ebx, 3
div ebx
mov edi,eax
mov eax, div
call sprint
mov eax, edi
call sprint
mov eax,edi
call iprintLF
mov eax, rem
call sprintdd
mov eax, edx
call iprintLF
call quit

^G Get Help ^O WriteOut ^R Read File ^Y Prev Pg ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where is ^V Next Pg ^U UnCut Text ^T To Spell
```

Рис. 4.10: Третья программа

Программа выполняет арифметические вычисления, на вывод идет результирующее выражения и его остаток от деления (рис. 4.11).

```
lab06 — -zsh — 80x24
tima.alekseev2109gmail.com@Mac lab06 % gcc lab6-3.c -o lab6-3
tima.alekseev2109gmail.com@Mac lab06 % ./lab6-3
Результат: 1
Остаток от деления: 4
tima.alekseev2109gmail.com@Mac lab06 %
```

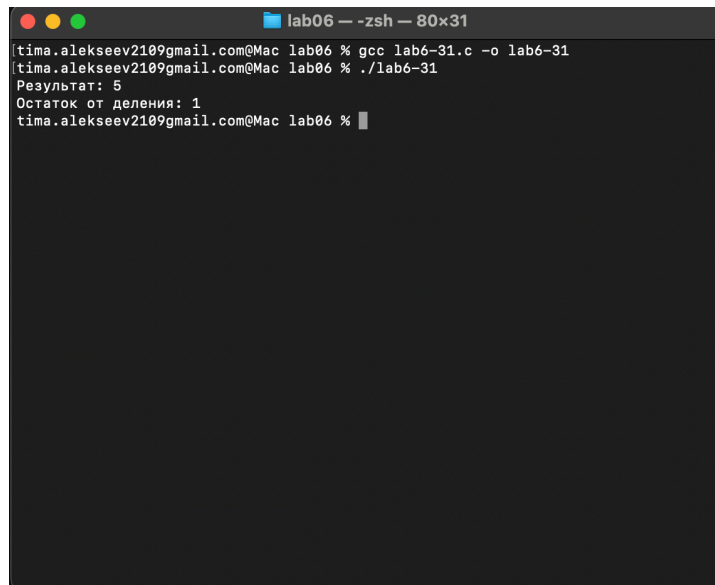
Рис. 4.11: Запуск третьей программы
Заменив переменные в программе для выражения $f(x) = (4*6+2)/5$ (рис. 4.12).

```
lab06 — nano lab6-3.asm — 80x31
UW PICO 5.09 File: lab6-3.asm Modified

#include 'in_out.asm'
SECTION .data
div: DB 'Результат: ', 0
rem: DB 'Остаток от деления: ', 0
SECTION .text
GLOBAL _start
_start:
mov eax, 4
mov ebx, 6
mul ebx
add eax, 2
xor edx,edx
mov ebx, 5
div ebx
mov edi,eax
mov eax, div
call sprint
mov eax, edi
call sprint
mov eax,edi
call iprintLF
mov eax, rem
call sprintdd
mov eax, edx
call iprintLF
call quit

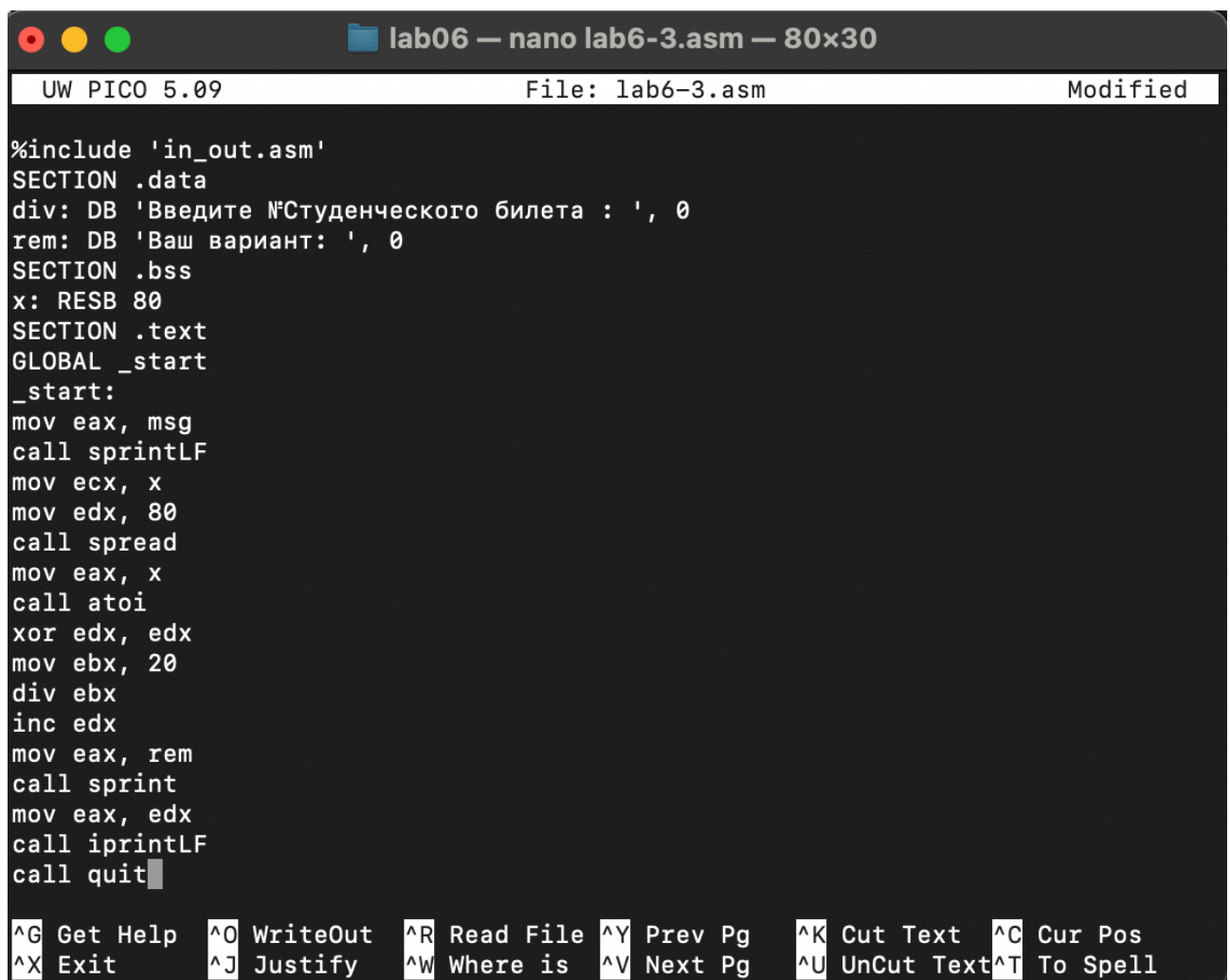
^G Get Help ^O WriteOut ^R Read File ^Y Prev Pg ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where is ^V Next Pg ^U UnCut Text ^T To Spell
```


Рис. 4.12: Изменение третьей программы
Запуск программы дает корректный результат (рис. 4.13).



```
lab06 — zsh — 80x31
tima.alekseev2109gmail.com@Mac lab06 % gcc lab6-31.c -o lab6-31
tima.alekseev2109gmail.com@Mac lab06 % ./lab6-31
Результат: 5
Остаток от деления: 1
tima.alekseev2109gmail.com@Mac lab06 %
```

Рис. 4.13: Запуск измененной третьей программы
Создаю новый файл и помещаю текст из листинга (рис. 4.14).



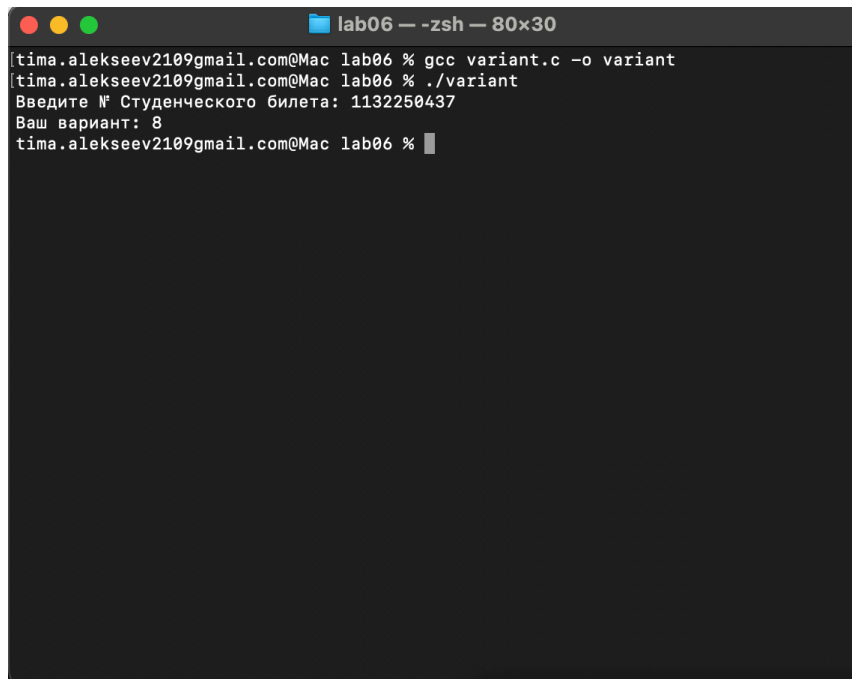
```
lab06 — nano lab6-3.asm — 80x30
UW PICO 5.09 File: lab6-3.asm Modified

#include 'in_out.asm'
SECTION .data
div: DB 'Введите №Студенческого билета : ', 0
rem: DB 'Ваш вариант: ', 0
SECTION .bss
x: RESB 80
SECTION .text
GLOBAL _start
_start:
mov eax, msg
call sprintLF
mov ecx, x
mov edx, 80
call spread
mov eax, x
call atoi
xor edx, edx
mov ebx, 20
div ebx
inc edx
mov eax, rem
call sprint
mov eax, edx
call iprintLF
call quit

^G Get Help ^O WriteOut ^R Read File ^Y Prev Pg ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where is ^V Next Pg ^U UnCut Text ^T To Spell
```


Рис. 4.14: Программа для подсчета варианта

Запустив программу и указав свой номер студенческого билета, я получил свой вариант для дальнейшей работы. (рис. 4.15).



```
lab06 — -zsh — 80x30
tima.alekseev2109gmail.com@Mac lab06 % gcc variant.c -o variant
tima.alekseev2109gmail.com@Mac lab06 % ./variant
Введите № Студенческого билета: 1132250437
Ваш вариант: 8
tima.alekseev2109gmail.com@Mac lab06 %
```

Рис. 4.15: Запуск программы для подсчета варианта

4.3 Ответы на контрольные вопросы

1. За вывод сообщения “Ваш вариант” отвечают строки кода:

- *mov eax,rem*
- *call sprint*

2. Инструкция *mov ecx, x* используется, чтобы положить адрес вводимой строки *x* в регистр *ecx* *mov edx, 80* - запись в регистр *edx* длины вводимой строки *call sread* - вызов подпрограммы из внешнего файла, обеспечивающей ввод сообщения с клавиатуры.

3. *call atoi* используется для вызова подпрограммы из внешнего файла, которая преобразует *ascii*-код символа в целое число и записывает результат в регистр *eax*.

4. За вычисления варианта отвечают строки:

- *xor edx,edx* ; обнуление *edx* для корректной работы *div*
- *mov ebx,20* ; *ebx = 20*
- *div ebx* ; *eax = eax/20*, *edx* - остаток от деления
- *inc edx* ; *edx = edx + 1*

5. При выполнении инструкции *div ebx* остаток от деления записывается в регистр *edx*.

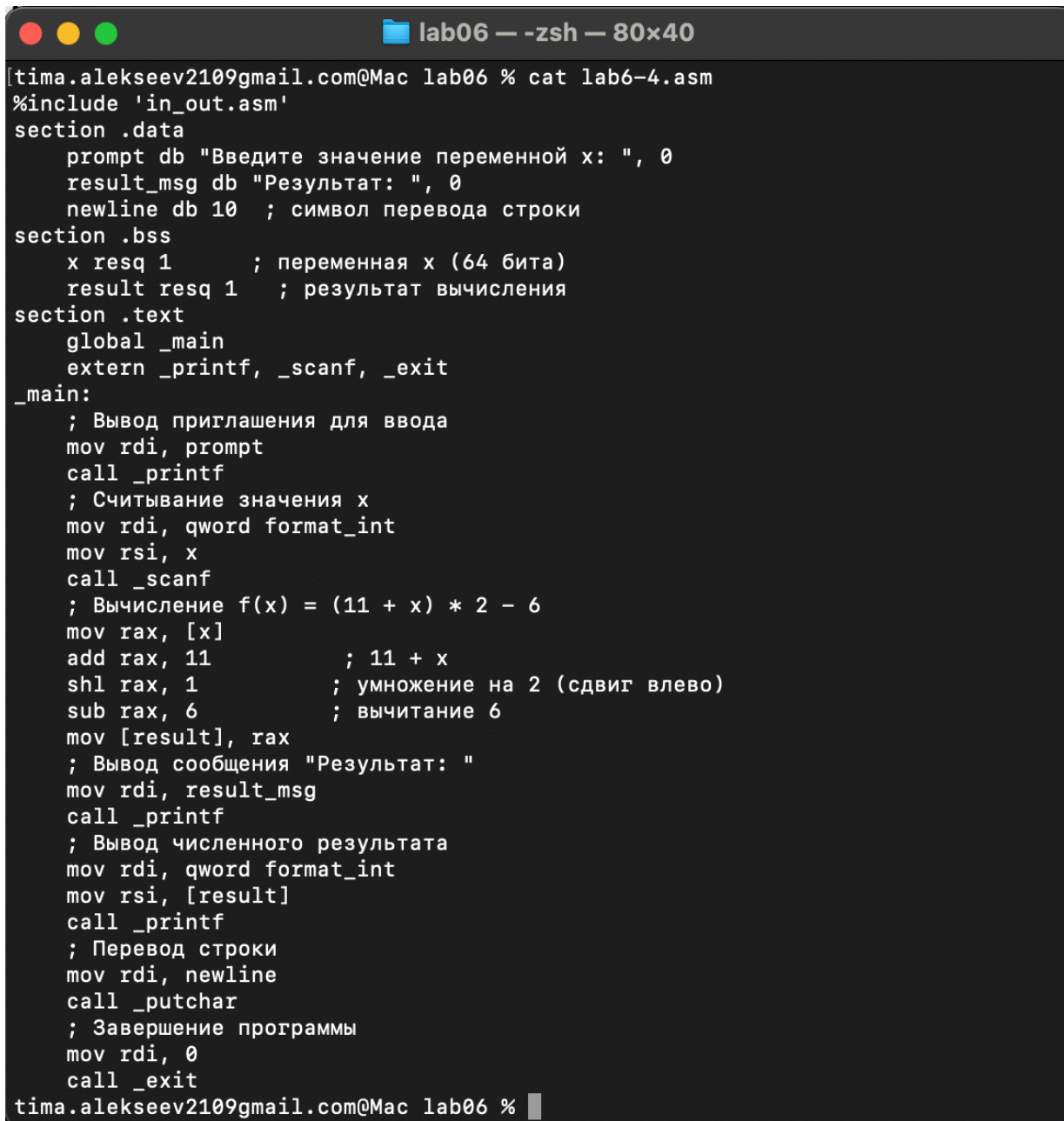
6. Инструкция *inc edx* увеличивает значение регистра *edx* на 1.

7. За вывод на экран результатов вычислений отвечают строки:

- *mov eax,edx*
- *call iprintLF*

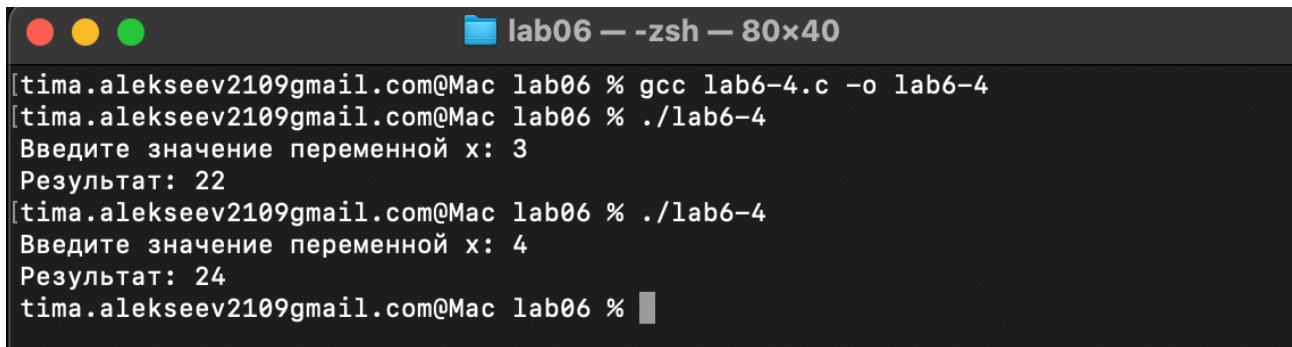
4.4 Задание для самостоятельной работы

В соответствии с выбранным вариантом, я реализую программу для подсчета функции $f(x) = (11+x)*2-6$, проверка на нескольких переменных показывает корректное выполнение программы (рис. 4.16).



```
lab06 — -zsh — 80x40
tima.alekseev2109gmail.com@Mac lab06 % cat lab6-4.asm
%include 'in_out.asm'
section .data
    prompt db "Введите значение переменной x: ", 0
    result_msg db "Результат: ", 0
    newline db 10 ; символ перевода строки
section .bss
    x resq 1 ; переменная x (64 бита)
    result resq 1 ; результат вычисления
section .text
    global _main
    extern _printf, _scanf, _exit
_main:
    ; Вывод приглашения для ввода
    mov rdi, prompt
    call _printf
    ; Считывание значения x
    mov rdi, qword format_int
    mov rsi, x
    call _scanf
    ; Вычисление f(x) = (11 + x) * 2 - 6
    mov rax, [x]
    add rax, 11 ; 11 + x
    shl rax, 1 ; умножение на 2 (сдвиг влево)
    sub rax, 6 ; вычитание 6
    mov [result], rax
    ; Вывод сообщения "Результат: "
    mov rdi, result_msg
    call _printf
    ; Вывод численного результата
    mov rdi, qword format_int
    mov rsi, [result]
    call _printf
    ; Перевод строки
    mov rdi, newline
    call _putchar
    ; Завершение программы
    mov rdi, 0
    call _exit
tima.alekseev2109gmail.com@Mac lab06 %
```

Прилагаю код своей программы:



```
lab06 — -zsh — 80x40
tima.alekseev2109gmail.com@Mac lab06 % gcc lab6-4.c -o lab6-4
tima.alekseev2109gmail.com@Mac lab06 % ./lab6-4
Введите значение переменной x: 3
Результат: 22
tima.alekseev2109gmail.com@Mac lab06 % ./lab6-4
Введите значение переменной x: 4
Результат: 24
tima.alekseev2109gmail.com@Mac lab06 %
```

Рис. 4.16: Запуск и проверка программы

5. Выводы

При выполнении данной лабораторной работы я освоил арифметические инструкции языка ассемблера NASM.

6. Список литературы

1. Пример выполнения лабораторной работы
2. Курс на ТУИС
3. Лабораторная работа №6
4. Программирование на языке ассемблера NASM Столяров А. В